

UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2023/2024

SEPTEMBER EXAMINATION

UCCD2044 OBJECT-ORIENTED PROGRAMMING PRACTICES

FRIDAY, 6 OCTOBER 2023

TIME : 2.30 PM – 4.30 PM (2 HOURS)

BACHELOR OF COMPUTER SCIENCE (HONOURS)
BACHELOR OF INFORMATION SYSTEMS (HONOURS)
BUSINESS INFORMATION SYSTEMS
BACHELOR OF INFORMATION SYSTEMS (HONOURS)
INFORMATION SYSTEMS ENGINEERING
BACHELOR OF INFORMATION TECHNOLOGY (HONOURS)
COMMUNICATIONS AND NETWORKING
BACHELOR OF INFORMATION TECHNOLOGY (HONOURS)
COMPUTER ENGINEERING

Instruction to Candidates:

This question paper consists of THREE (3) questions in Section A and TWO (2) questions in Section B.

Answer **ALL** questions in **Section A** and **ONLY ONE (1)** question in **Section B**.
Each question carries 25 marks.

Should a candidate answer more than ONE (1) questions in Section B, marks will only be awarded for the FIRST questions in that section in the order the candidate submits the answers.

No calculator is allowed for this paper.

Answer questions only in the answer booklet provided.

UCCD2044 OBJECT-ORIENTED PROGRAMMING PRACTICES**SECTION A**

- Q1. (a) The JavaFX and Event Handling questions.
- (i) What methods do you call to register mouse event handler for mouse pressed, mouse clicked, and mouse dragged event? (3 marks)
 - (ii) By using a scenario as an example, explain when you would use a **TextField** instead of **TextArea** in JavaFX. (5 marks)
- (b) Fill in the missing codes in **Figure 1(a)** to produces the **layout as shown in Figure 1(b)**. (5 marks)

```

public void start(Stage primaryStage) {

    Button vegies = new Button("Vegies");
    Button rice = new Button("Rice");

    TextArea text = new TextArea("This is a text
area");

    /* create layout and set root here */

    Scene scene = new Scene(root, 300, 250);
    primaryStage.setScene(scene);
    primaryStage.show();
}

```

Figure 1(a) Codes that produce the output layout in **Figure(b)**

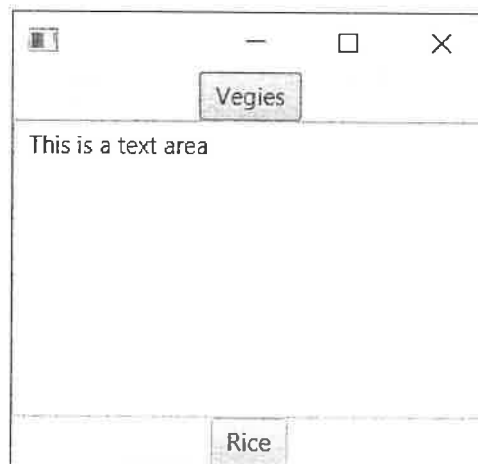


Figure 1(b) Output produce from codes in **Figure1(a)**

UCCD2044 OBJECT-ORIENTED PROGRAMMING PRACTICES**Q1. (Continued)**

- (c) **Figure 2(a)** shows a simple JavaFX application that uses **three** buttons to:
- **Change color for the pie chart** (your preferences for the colors)
 - **Change color for the fonts in the pie chart** (your preferences for the colors)
 - **Move the pie chart and fonts to the right by 10 pixels** (for each click)

Complete the program in **Figure 2(b)** by filling out the missing codes for parts: **(i)**, **(ii)**, **(iii)**, and **(iv)**. You may omit the **import** statement in your answer.

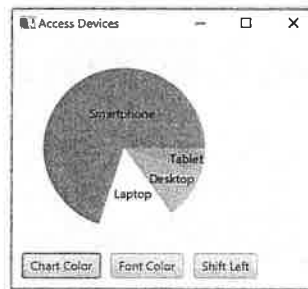


Figure 1. The Initial Screen.

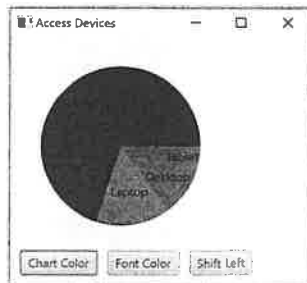


Figure 4. Button "Chart Color" is pressed.

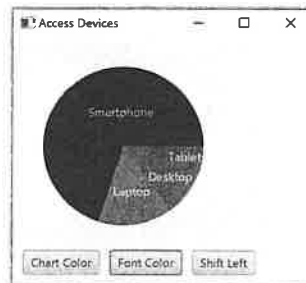


Figure 3. Button "Font Color" is pressed.

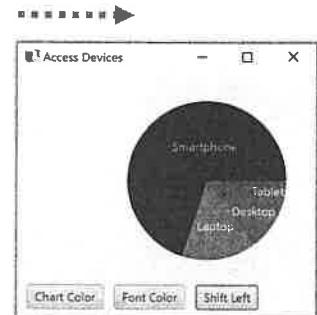


Figure 2. Button "Shift Left" is pressed.

Figure 2(a) Example application with three buttons to change colors and font colors in pie chart and move the pie chart

```
public class Question_1c extends Application {

    public void start(Stage primaryStage) {

        // Pane pane for placing the arc
        Pane pane = new Pane();
        Arc[] arcs = new Arc[4];
        Text[] txts = new Text[4];

        // first arc
        arcs[0] = new Arc(100, 100, 80, 80, 0, 360 * 0.7); // 70 %
        arcs[0].setFill(Color.LIGHTCORAL);
        arcs[0].setType(ArcType.ROUND);
        txts[0] = new Text(65, 70, "Smartphone");

        // refer to Q1(c) (i)
```

UCCD2044 OBJECT-ORIENTED PROGRAMMING PRACTICES**Q1. (c) (Continued)**

```

        // HBox pane for holding the three buttons
        HBox hBox = new HBox();
        hBox.setSpacing(10);
        hBox.setAlignment(Pos.BASELINE_LEFT);
        Button btChartCol = new Button("Chart Color");
        Button btFontCol = new Button("Font Color");
        Button btPosition = new Button("Shift Left");
        hBox.getChildren().addAll(btChartCol, btFontCol,
        btPosition);

        // refer to Q1(c) (ii)

        // Create a border pane and place pane and hBox in the
border pane
        BorderPane borderPane = new BorderPane();
        borderPane.setCenter(pane);
        borderPane.setPadding(new Insets(10));
        borderPane.setBottom(hBox);
        BorderPane.setAlignment(hBox, Pos.BOTTOM_CENTER);

        // refer to Q1(c) (iii)

    }

    // refer to Q1(c) (iv)
}

```

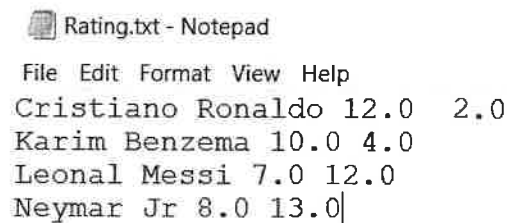
Figure 2(b) Program to generate the application in **Figure 2(a)**

- (i) Add the rest of the arcs with 15% for Laptop, 10% for Desktop and 5% for Tablet. (3 marks)
- (ii) Add the event handling code for the three buttons using *Lambda* expression. Perform the respective actions as mentioned above (refer to **Figure 2(a)** when these buttons are pressed. (6 marks)
- (iii) Create a scene of size (300,250) to host the border pane, place the scene into the stage and show the stage. (2 marks)
- (iv) Add `main()` to launch the application. (1 mark)

[Total : 25 marks]

UCCD2044 OBJECT-ORIENTED PROGRAMMING PRACTICES

- Q2. (a) The following questions are based on File Input and Output,
- (i) Describe and show a simple example on how to use **FileInputStream**.
(2 marks)
 - (ii) What are the main differences between **text I/O** and **binary I/O**?
(2 marks)
- (b) A text file “Rating.txt” stores footballers’ first name, last name, goal (double) and assist (double), as shown below:



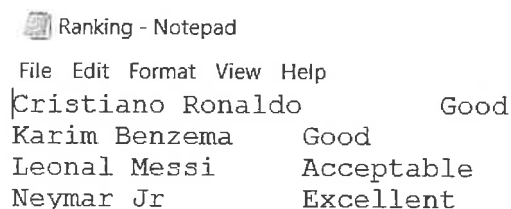
```

File Edit Format View Help
Cristiano Ronaldo 12.0 2.0
Karim Benzema 10.0 4.0
Leonel Messi 7.0 12.0
Neymar Jr 8.0 13.0

```

Create a complete Java program to perform the following tasks describe in (i), (ii) and (iii). Write your answer in a single main method. You may omit the **import** statement in your answer.

- (i) Read from “Rating.txt” and compute the final score (score = $0.6 \times \text{goal} + 0.4 \times \text{assist}$ for each player.
(6 marks)
- (ii) Use the final score to assign a rating for each player. There is four possible rating:
Excellent ($8.0 \leq \text{score}$), Good ($6.0 \leq \text{score} < 8.0$), Acceptable ($4.0 \leq \text{score} < 6.0$), and Worst ($\text{score} < 4.0$).
(3 marks)
- (iii) Output the name and final score of each of the player to a text file named “Ranking.txt”. For example, using the sample records shown above, after execution of your program the content of output file should be:



```

File Edit Format View Help
Cristiano Ronaldo Good
Karim Benzema Good
Leonel Messi Acceptable
Neymar Jr Excellent

```

(2 marks)

UCCD2044 OBJECT-ORIENTED PROGRAMMING PRACTICESQ2. (Continued)

- (c) The following question is based on Java Database Programming.
- (i) What are the advantages of developing database applications using JDBC? (2 marks)
 - (ii) Given the URL "jdbc:ucanaccess://Drive:/DatabaseDir/DatabaseLab" for an Access database that contains a table, named **Salary** (**id int**, **firstName varchar(50)**, **lastName varchar(50)**, **dept varchar(15)**, **salary float**). Write a complete Java program (JDBC) to connect to the database, query results, and display all the column names and contents resulting from the **Salary table**. You need to include import statement in your coding. Sample output is shown below. (8 marks)

Output:

```

Successfully connected to database
Table creation process successfully!
1      Paul    Chen    Sale    3000.0

```

[Total : 25 marks]

Q3. Answer the following questions related to exception handling and multithreading programming.

- (a) Explain the difference between a **checked** exception and an **unchecked** exception. (2 marks)
- (b) Briefly explain how to handle an exception in a program. Provide a simple coding as an example. (4 marks)
- (c) Rewrite Q3_c method below so that it catches the **IllegalArgumentException** thrown by divide and prints the message "The divisor is zero" if it is thrown. (6 marks)

```

public static void Q3_c(double m, double n) {

    /* Write code use try-catch block with correct syntax
    * catches the IllegalArgumentException and print the
    * correct message. */

    System.out.println(divide(m, n));

}

```

UCCD2044 OBJECT-ORIENTED PROGRAMMING PRACTICES**Q3. (Continued)**

(d) What is a thread and why is multithreading needed? (2 marks)

(e) Given the code segment from a threaded program as shown below, write the code to start a thread. (3 marks)

```
class Demo implements Runnable {

    public void run() {

        System.out.println("Thread is in Running state");

    }

    public static void main(String args[]) {

        // Code to start a thread

    }

}
```

(f) Identify and correct the error in the following program. (2 marks)

```
1. public class FirstThread implements Runnable {
2.     public static void main(String[] args) {
3.         new FirstThread();
4.     }
5.     public FirstThread() {
6.         new Thread(task).run();
7.     }
8.     public void run() {
9.         System.out.println("test");
10.    }
11. }
```

(g) The following codes is a segment of codes from a multithreaded program. Is it a thread-safe class? Explain your answer. (3 marks)

```
public class Counter {
    private static int count = 0;

    public int getCount(){
        return count++;
    }

}
```

(h) How to avoid race condition in a multithreaded program? (3 marks)

[Total : 25 marks]

UCCD2044 OBJECT-ORIENTED PROGRAMMING PRACTICES**SECTION B**

This section consists of TWO (2) questions. Answer **ONLY ONE (1)** question.

- Q4. (a) List the differences between abstract class and interface in terms of data fields, constructors, and methods. (6 marks)
- (b) Explain what is encapsulation and how it is implemented in Java. (3 marks)
- (c) Answer the following sub-questions based on the Java class below:

```

1. public class Q4_c {
2.     public static void main(String[] args) {
3.         int[] arr = {1,2,3,4,5};
4.         MyArray myArr = new MyArray(arr);
5.         arr[0] = 15;
6.         System.out.println(arr[0]);
7.         System.out.println(myArr.getItem(0));
8.     }
9. }
10.
11. class MyArray {
12.     private int[] array;
13.     public MyArray(int[] arr) {
14.         array = arr;
15.     }
16.     public int getItem(int index) {
17.         return array[index];
18.     }
19. }

```

- (i) What are the outputs of lines 6 and 7? Explain why the results are achieved and how it violates the concept of encapsulation. (4 marks)
- (ii) Rewrite the constructor of MyArray class to prevent such violation from happening. (3 marks)
- (iii) What are the outputs of lines 6 and 7 after rewriting the constructor of MyArray class in Q4(c)(ii)? (2 marks)
- (d) Create a Circle class to represent the circle objects. The class contains the following requirements:
- (i) Two private data fields named radius of double type and color of String type. (1 mark)

UCCD2044 OBJECT-ORIENTED PROGRAMMING PRACTICES**Q4. (c) (Continued)**

- (ii) A no-arg constructor that set the radius and color to default values of 1 and "red" respectively and the second constructor is used to initialize the two data fields. (4 marks)
 - (iii) A public method named `getArea()` that returns the area of this circle. (2 marks)
- [Total : 25 marks]

Q5. (a) Create a JAVA class named **Account**, which models a bank account. The class contains the following attributes and methods:

- (i) Three data fields: `accountNumber` (int), `balance` (double) which maintains the current account balance, and `annualInterestRate` (double) that stores the current interest rate. Assume all accounts have the same interest rate. (3 marks)
- (ii) A constructor that creates a default account. (2 marks)
- (iii) A constructor that creates an account with the specified `accountNumber` and assign a default balance to 0.0. (2 marks)
- (iv) Accessor and mutator methods for the data fields. There is no mutator for `accountNumber` as it is not designed to be changed. (3 marks)
- (v) Two instance methods, named `credit()` and `debit()`, which adds/subtracts the given amount to/from the balance, respectively. Check if there is sufficient balance for debit method (available balance should be greater than current balance). (3 marks)
- (vi) A method named `getMonthlyInterestRate()` that returns the monthly interest rate. (2 marks)
- (vii) `toString()` method which returns "A/C no: xxx, Balance = xxx.xx, Monthly Interest = xxx.xx", with balance and monthly interest are rounded to two decimal places. (2 marks)

(b) Write the **AccountTest** class and complete the following sub-questions:

- (i) Test constructor, mutator, and use `toString()` to display the output as below.

A/C no:1234, Balance=10000.00, Monthly Interest Rate=0.02

(2 marks)

UCCD2044 OBJECT-ORIENTED PROGRAMMING PRACTICES**Q5. (b) (Continued)**

- (ii) Test `debit ( )` and use `toString ( )` to display the following output.

The requested amount exceeds your current balance!

A/C no:1234, Balance=10000.00, Monthly Interest Rate=0.02

(2 marks)

- (c) Draw the **UML** class diagram for Account. Provide details of all the data fields and methods.

(4 marks)

[Total : 25 marks]
